

Warp-Resident NTT and Keccak Collectives for ML-KEM on a RISC-V GPU

Anonymous Authors

Abstract—ML-KEM combines polynomial butterflies with Keccak reductions and permutations, creating different communication demands within one cryptographic request. We implement collective instructions that operate on ordinary lane registers of a RISC-V SIMT GPU and retain single-destination writeback. NTT uses XOR-paired butterflies and a shared 16-multiplier bank; Keccak uses a 25-lane state with stage or whole-round operations. On a common RV32IM core, cycle-accurate XRT simulation shows that stage and round instructions accelerate complete ML-KEM-768 by $1.462\text{--}1.552\times$ and $1.520\text{--}1.583\times$, respectively, over the same software shuffle mapping at one and eight requests. Round fusion improves the expanded stage permutation by $3.29\times$, but a separate matched SimX control shows only about 3% improvement for complete requests. Independently routed stage and round extensions add 1.129% and 0.898% core LUTs over an NTT-equipped baseline at 250 MHz. These results show why collective granularity must be selected using complete requests and concurrency, rather than primitive instruction count alone.

Index Terms—ML-KEM, Keccak, NTT, SIMT, RISC-V, instruction-set extension

I. INTRODUCTION

ML-KEM standardizes a module-lattice key-encapsulation mechanism whose implementation combines number-theoretic transforms (NTTs), polynomial arithmetic, sampling, and SHA-3/SHAKE [1], [2]. These operations expose substantial parallelism, but their communication differs. An NTT butterfly exchanges a pair of coefficients and applies a modular product. A Keccak round reduces columns of a 5×5 state, permutes words, and combines neighboring words. Mapping both to independent scalar lanes leaves structured communication to software; sending complete operations to a memory-coupled engine introduces a different state-transfer boundary.

An extensible SIMT processor permits a third option: keep the state in the warp’s ordinary registers and expose the required communication through collective instructions. However, register residence alone does not determine the useful instruction size. A larger operation reduces issue demand but requires more pipeline state, while its application benefit depends on the remaining sponge, arithmetic, and control work. Concurrent requests can also change which execution interface is fastest.

We investigate this design space on Vortex, an open RISC-V GPU [3]. Our design distinguishes *where state resides*, *which lanes cooperate*, and *how much computation one instruction performs*. For NTT, a strided register layout separates three local levels from four cross-lane levels. A pair collective computes one modular product and returns the two butterfly outputs through the lanes’ normal destinations. For Keccak,

a 25-lane layout supports either three stage operations or a complete round. RV32 explicitly selects low and high output halves; RV64 returns whole words. Neither form requires persistent hidden accelerator state between instructions.

The goal is a realizable SIMT interface, not a claim that these state layouts or whole-round arithmetic are new. CPU register-coupled cryptographic extensions already exist [4], [5]; ML-Cube uses 25-thread Keccak with shuffles [6]; and a published patent application describes warp-collective Keccak rounds [7]. We instead make the operand, participation, and writeback contracts explicit, implement them in processor RTL, and evaluate their resource and application tradeoffs.

This paper makes three contributions:

- 1) A register interface that fits multi-input/multi-output cryptographic operations into ordinary SIMT operands: pair-low twiddle selection and distributed butterfly outputs for NTT, and explicit output-half selection for 25-lane Keccak, all with one destination per lane.
- 2) Data paths matched to these structures: a statically routed NTT unit sharing 16 multipliers across 32 lanes, and stage/round Keccak pipelines using the existing operand and result interfaces.
- 3) A controlled evaluation separating software mapping, instruction acceleration, fusion, and batching. Complete ML-KEM reveals diminishing fusion returns and a latency/throughput crossover with a pointer engine; post-route implementation quantifies the cost of each register interface.

II. COMMUNICATION AND STATE PLACEMENT

A. Two primitives, two participation patterns

For the ML-KEM modulus $q = 3329$, a 256-coefficient forward NTT has seven levels, with butterfly distances 128, 64, 32, 16, 8, 4, 2. A 32-thread warp distributes the polynomial as

$$R_k[l] = a[l + 32k], \quad 0 \leq l < 32, \quad 0 \leq k < 8. \quad (1)$$

Figure 1 contrasts the two state layouts and their communication. At the first three levels, partners occupy different registers in the same lane. At the remaining levels, the partner is lane $l \text{ xor } d$ in the same register vector. There is no distance-one level in ML-KEM. Contiguous loads/stores for each k surround the transform; the computation can retain all eight coefficients in lane registers between levels.

Keccak-f[1600] instead has 25 words of 64 bits and 24 rounds. Lane $l = x + 5y$ holds $A[x, y]$, $0 \leq x, y < 5$.

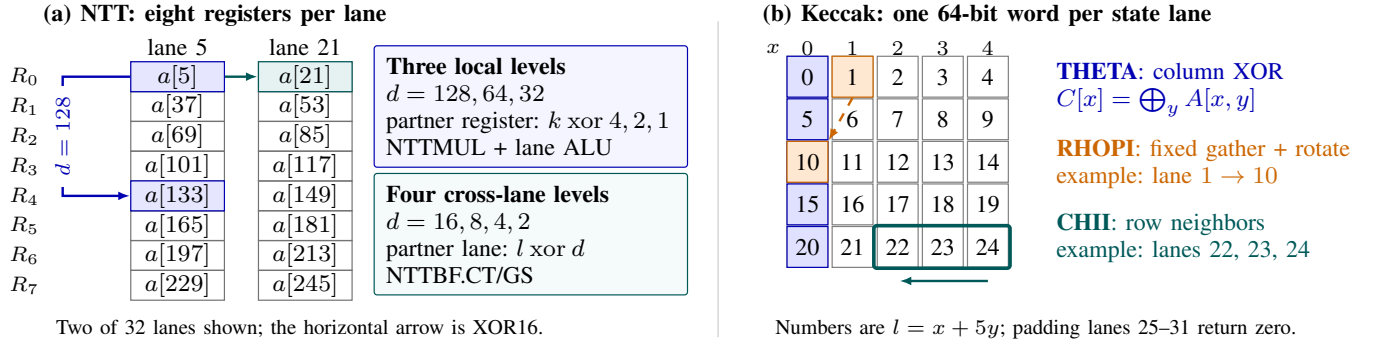


Fig. 1. The communication structure determines the collective’s scope. (a) NTT changes register index at its first three levels and lane index at its last four. (b) Keccak combines column reduction, fixed gather/rotation, and row-neighbor logic on a different lane layout. RV32 stores each state word as low/high GPR values.

Its column parity, fixed word permutation, and row-neighbor dependencies involve different lane sets from the NTT. Lanes 25–31 contain no state, so only 25/32 of the lane positions represent words. On RV32, each word occupies two GPR values; on RV64, one. These are state-storage counts, not the peak temporary-register requirement.

B. Why both software and hardware boundaries matter

Three baselines answer different questions. Scalar C and optimized PQRV assembly [8] establish the cost of assigning Keccak to one lane. A software 25-lane shuffle implementation separates the benefit of the cooperative mapping from the benefit of new instructions. A pointer engine accepts a 200-byte state address, executes the complete permutation, and writes the state back through the memory path. It tests a coarser execution boundary rather than a different implementation of the same register opcode.

Software cooperation is not uniformly best when implemented with shuffles. For NTT, a software mapping that groups levels as $3 + 3 + 1$ with shared-memory transposes is stronger than the direct-shuffle version in our archived pre-extension tests. We retain this denominator when evaluating NTT, rather than assigning all improvements over a weaker mapping to hardware. Similarly, one Keccak state per warp is a controlled occupancy point; scalar and five-lane software can pack more independent states into a warp.

III. REGISTER COLLECTIVE ARCHITECTURE

Figure 2 relates the layouts to their execution paths. The arithmetic units are separate. They share the processor’s ordinary operand/result interfaces, not a common modular/Boolean data path or a demonstrated physical implementation of the general shuffle network.

A. Paired NTT with one result per lane

NTTMUL.K performs signed narrow Montgomery multiplication, with $R = 2^{16}$ and modulus 3329. It supports the three lane-local NTT levels, inverse scaling, and the products in cooperative polynomial arithmetic. For a cross-lane operation, let p denote the pair-low lane and $h = p \text{ xor } 2^s$ the pair-high lane. The first source vector supplies a and b at these

positions; the second source at p supplies the twiddle ζ . The CT collective computes

$$t = \text{Mont}(b\zeta), \quad (r_p, r_h) = (a + t, a - t). \quad (2)$$

Only one modular product is needed for the pair. Both lanes write their own destination, resolving a butterfly’s three-input/two-output structure without a multi-destination instruction. The GS inverse form implements the library’s sign convention:

$$(r_p, r_h) = (\text{Barrett}(a + b), \text{Mont}((b - a)\zeta)). \quad (3)$$

Sums and differences wrap to signed 16 bits before reduction, matching the reference’s finite-width behavior. The equations do not imply unbounded intermediate integers.

The instruction encodes $s \in \{0, 1, 2, 3, 4\}$. ML-KEM uses $s = 4, 3, 2, 1$, eight instructions per cross-lane level and 32 per transform direction. Both members of an enabled pair must participate; a partial mask is legal when every selected pair is either fully active or fully inactive. The supported data path has 32 ALU lanes and converged participating threads; narrower packetized ALUs would require additional operand collection and are outside this implementation.

Figure 3 shows the concrete CT operation at distance 16. Lane 5 owns coefficient $a[5 + 32k]$ and the twiddle, while lane 21 owns $a[21 + 32k]$. The collective routes both operands to one pair computation and distributes its results. The pair-high twiddle is ignored; this is an architectural rule, not a compiler assumption that both twiddle operands happen to agree. Tests supply different per-lane twiddles to expose violations.

The final unit assigns each pair to one of 16 signed 16×16 multipliers. Static routing for the five encoded XOR distances avoids implementing an arbitrary partner permutation. Pair CT uses one product per bank element. GS reuses the bank first for $(b - a)\zeta$ and then for the Barrett-quotient product $(a + b)20159$. Lane-local multiplication processes the lower and upper 16-lane halves in consecutive phases. With registered reduction and result stages, CT has direct latency/initiation interval 6/1 cycles; GS and a full W32 NTTMUL.K have 7/2. The result interface preserves the original lane positions and metadata

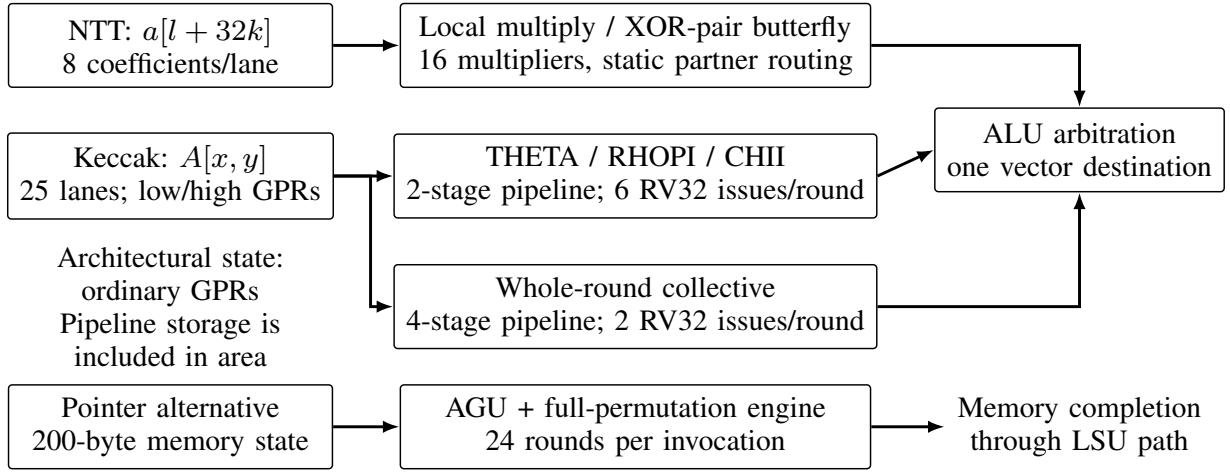


Fig. 2. Communication-specific execution boundaries. NTT uses lane-local and pair operations; Keccak Stage and Round are alternative collective granularities. The pointer baseline transfers state through memory. Issue counts are architectural operations, not dependent instruction cycles.

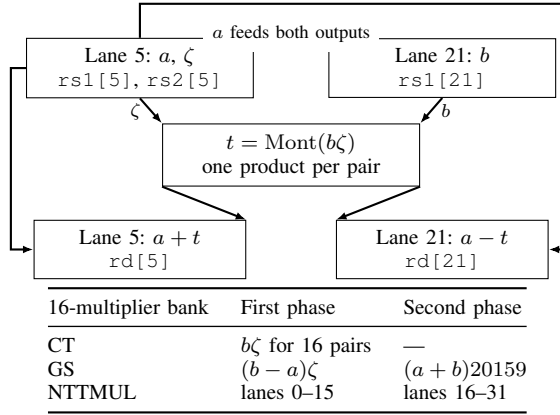


Fig. 3. NTT pair computation and bank reuse. At XOR16, lanes 5 and 21 each write their own destination using the pair-low twiddle. Sixteen such pairs cover W32. CT needs one bank phase; GS and NTTMUL need two. Phases describe multiplier admission, not complete instruction latency.

under backpressure. These unit-level figures do not include core scheduling, operand hazards, or result arbitration.

B. Stage and round Keccak without hidden state

The Stage design exposes THETA, RHOPI, and CHII/IOTA. Writing all coordinates modulo five, its communication is

$$C[x] = \bigoplus_{y=0}^4 A[x, y], \quad (4)$$

$$T[x, y] = A[x, y] \oplus C[x - 1] \oplus \text{ROL}_{64}(C[x + 1], 1), \quad (5)$$

$$B[y, 2x + 3y] = \text{ROL}_{64}(T[x, y], \rho[x, y]), \quad (6)$$

$$U[x, y] = B[x, y] \oplus (\neg B[x + 1, y] \wedge B[x + 2, y]). \quad (7)$$

IOTA XORs the round constant into lane zero. THETA needs a column reduction; RHOPI needs a fixed gather and rotation; CHII needs two neighboring row words. Fixed connections implement these dependencies directly.

On RV32, THETA and RHOPI read the old low/high state vectors and return one selected half. The rotation crosses the half-word boundary, so two independent 32-bit operations would be incorrect. CHII operates on one half at a time and takes a uniform round index as its second source. Six instructions implement one round. Both halves of a stage must read the same old state, so temporaries prevent the first destination from destroying an operand of the second instruction. The all-stage permutation loop uses five GPRs including control and temporaries and has no loads, stores, calls, or stack frame in its disassembly.

The stage unit uses two pipeline stages and has initiation interval one. The Round alternative fuses column parity, THETA, RHOPI, and CHII/IOTA into four registered stages, also with initiation interval one. Each RV32 `KROUND.L/H` instruction reads the complete old state and computes its selected output half; the pair recomputes from identical operands rather than retaining hidden state for a later high-half write. The literal round index is encoded in the instruction, producing 48 issues per permutation instead of Stage’s 144. Ordinary hazards and writeback arbitration remain.

Figure 4 shows the RV32 dependencies. Both output-half instructions read the same old state; the first must not overwrite an operand needed by the second. Each issue carries its own warp/destination metadata, so different warps may interleave without an implicit low/high transaction or private context bank. The RV64 comparison measures whether removing this half-word interface improves the application.

All 32 lanes must execute a Keccak collective at the same instruction with the full mask, including the seven padding lanes, whose outputs are zero. Guarding execution by `lane < 25` violates the contract. Loads and stores can be guarded if lanes reconverge before the collective. Stage’s round operand must be uniform and in $0, \dots, 23$; Round encodes that range. Simulation assertions and contract tests diagnose invalid participation.

RV64 changes the word interface, not the state distribution:

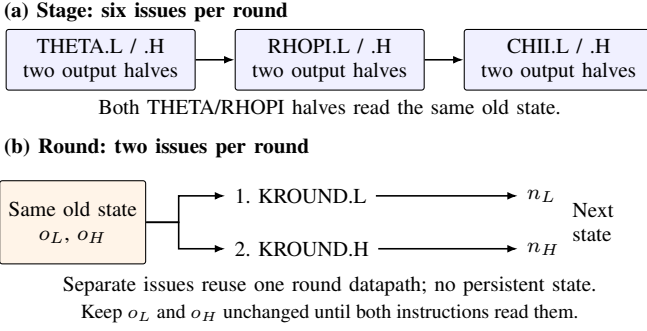


Fig. 4. RV32 issue dependencies for one Keccak round. L/H are separate issues through one shared unit, each with one vector destination. Stage uses six issues; Round uses two and recomputes from identical old operands. The diagram shows dependencies, not simultaneous issue or a cycle schedule.

TABLE I
REGISTER COLLECTIVES. L/H DENOTES SEPARATE RV32 OUTPUT-HALF INSTRUCTIONS. ALL OUTPUTS USE ONE DESTINATION PER PARTICIPATING LANE.

Operation	Communication	Output / RV32 issues
NTTMUL.K	Lane local	One modular product
NTTBF.CT/GS	XOR pair	One butterfly / 1
THETA.L/H	Five-word column	One stage / 2
RHOPI.L/H	Fixed word gather	One stage / 2
CHII.L/H	Row neighbors	One stage / 2
KROUND.L/H	Complete 25-lane state	One round / 2

THETA, RHOPI, and CHII return a complete 64-bit word, and Round needs one issue per round. The narrow ML-KEM multiplier and butterfly semantics remain unchanged. Table I summarizes the resulting contracts.

C. Integrating the complete request

The ML-KEM integration is based on mlkem-native [9]. Lane zero executes scalar control, while collective wrappers activate the warp for NTT, cooperative polynomial arithmetic, and FIPS-202. The wrappers broadcast arguments, perform the operation, and restore the leader mask. A request has a shared allocation arena, avoiding 32 redundant allocations of the same workspace. This integration cost is included in application timing, rather than eliminated by measuring only an isolated primitive.

One-shot sponge calls retain lane state across absorb, permutation, and squeeze. Incremental SHAKE saves its context on return and reloads it on the next call. Thus register residence eliminates transfers *between primitive stages*, not every state load/store in the complete library. The pointer alternative loads and stores the 200-byte state on each permutation invocation through its address-generation and memory path.

IV. EVALUATION

A. Method and comparison boundaries

The primary platform is one RV32IM core with eight resident warps, 32 threads per warp, and 32-wide ALU/LSU paths. F/D and L2/L3 are disabled; instruction cache, data cache,

and local memory are each 16 KiB. The unified application experiment enables NTT, Stage, Round, and Pointer in the *same* XRT runtime and changes only the software Keccak backend. NTT and cooperative polynomial arithmetic are held fixed.

Each request runs ML-KEM-768 KeyGen, Encaps, and Decaps using the same FIPS 203 known-answer-test inputs. All public/secret keys, ciphertexts, and encapsulation/decapsulation shared secrets match byte for byte. Each request performs 140 permutations, 15 NTTs, and nine inverse NTTs. We measure one request ($M = 1$) and eight concurrent requests ($M = 8$). For M8, we compare complete-batch launch cycles, not the sum of overlapping per-request intervals. Reported whole-launch device cycles include launch/teardown work and exclude host transfers.

XRT simulation exercises both the AFU and processor RTL; it is not a board measurement. The 12 unified application cases match SimX retired-instruction counts exactly, with maximum whole-launch cycle difference 2.710%. Primitive tests verify coefficients/words, guard regions, pair twiddle selection, active-mask contracts, and backpressure. Standalone butterfly validation checks 334,080 outputs across all five distances. Performance figures use their stated timing scope rather than inferring throughput from instruction counts.

B. Software mapping and complete-KEM gains

Figure 5 first shows why scalar software is an insufficient instruction baseline. Moving from PQRV to software SG25 changes M1 cycles from 24.672 million to 7.909 million and M8 from 37.753 million to 19.440 million, before any Keccak collective instruction is used. Relative to this same shuffle mapping, Stage improves complete requests by $1.462\times/1.552\times$ and Round by $1.520\times/1.583\times$ at M1/M8. Against PQRV, Round reaches $4.742\times/3.074\times$, but those larger ratios include the software mapping improvement. All these denominators already use accelerated NTT; they are not combined Keccak-plus-NTT gains over an unmodified GPU.

The preferred backend changes with concurrency. At M1, Round takes 5.203 million cycles versus Pointer’s 5.878 million, a $1.130\times$ advantage. At M8, Pointer finishes in 8.939 million cycles versus Round’s 12.280 million, a $1.374\times$ advantage in the opposite direction. Register residence therefore does not guarantee the best batch throughput. The two interfaces create different issue and memory traffic; the observed crossover establishes a design boundary, but these measurements alone do not assign it to one specific scheduler or cache mechanism.

C. Fusion improves the primitive more than the application

Round embeds a literal round index, so its code is expanded. Comparing it only with looped Stage would also count removed loop overhead. We therefore expand Stage’s 24 rounds and measure the same input/output-isolated interval with one state per warp and 64 dependent permutations per state. Figure 6 uses XRT results from that matched control. With eight active warps, expanded Stage takes 387.354 cycles per completed permutation and Round takes 117.693, a $3.291\times$

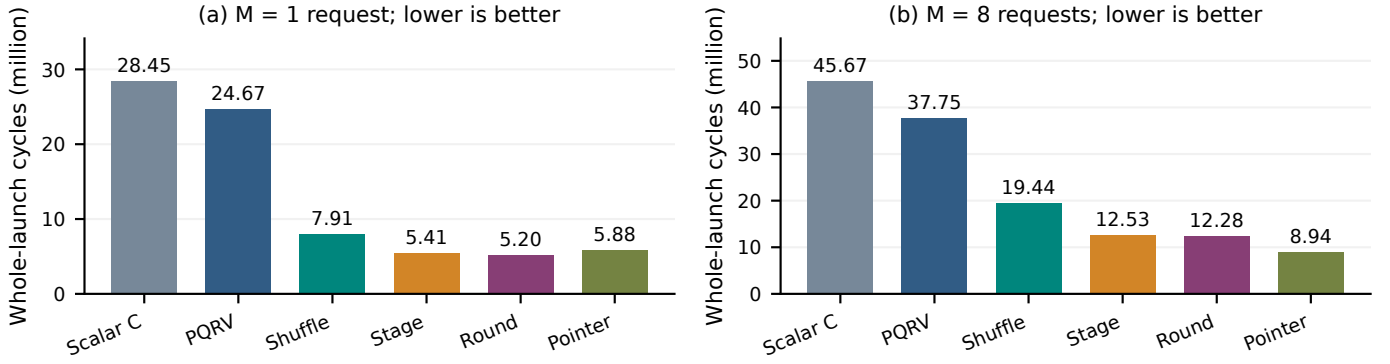


Fig. 5. Complete ML-KEM-768 on one common RV32IM XRT runtime, with the final NTT and cooperative arithmetic fixed. All requests pass KAT. Stage is looped and Round is expanded here; the separate equal-expansion control isolates fusion. M1 shows latency; M8 shows the time to complete eight requests.

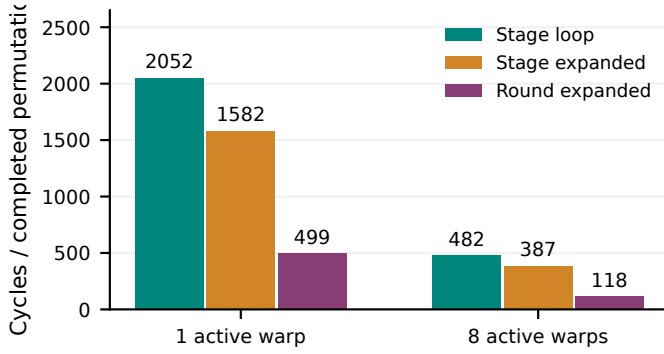


Fig. 6. Matched expansion control, XRT: 64 dependent permutations per state, one state per active warp. Values normalize the isolated span by completed permutations; they do not represent maximally packed software throughput.

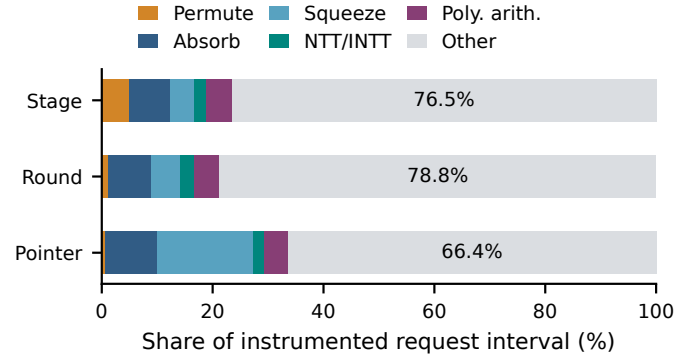


Fig. 7. Mutually exclusive M1 intervals from independently enabled RV32IM backends, measured through XRT. “Other” is a residual of the instrumented request, not one cryptographic primitive.

improvement. For one active warp, the corresponding ratio is $3.168\times$.

The application benefit is much smaller. In the matched expansion SimX control, M1 cycles change from 5,341,994 to 5,198,475 and M8 from 12,336,487 to 11,937,913: only $1.028\times$ and $1.033\times$. These controls use a Stage/Round-enabled core and are kept distinct from the all-backend XRT experiment. They show that the large primitive gain survives equal code expansion, but most request time lies elsewhere.

Separate M1 XRT profiles provide a direct explanation (Fig. 7). Permutation occupies 4.964% of the instrumented Stage request, 1.266% for Round, and 0.516% for Pointer. Absorb and squeeze exclude nested permutation intervals; NTT/INTT contributes approximately 2.1–2.4%. Remaining work includes sampling, encoding, comparisons, memory, control, wrappers, and instrumentation. The matched SimX control estimates probe overhead at 6.6–8.3%, so these intervals explain bottleneck migration but do not replace the uninstrumented speedup denominator. Further permutation fusion cannot remove this remaining work.

For scale, an Amdahl-style calculation with Stage’s instrumented fraction $f = 0.04964$ and single-warp primitive speedup $s = 3.168$ gives $1/(1 - f + f/s) \approx 1.035$. This illustrates the small remaining opportunity; it is not an unin-

strumented prediction. At M8, overlapping warps and shared resources also prevent adding single-request fractions into a batch model.

D. NTT communication and multiplier-bank sizing

We separate the NTT study from the final Keccak sweep because it uses an earlier matched pointer-enabled core. Holding the full-bank modular multiplier fixed, replacing software shuffles with pair butterflies reduces complete-KEM SimX cycles by 2.564% at M1 and 8.232% at M8. This comparison isolates the communication instruction. Against the stronger shared-memory $3 + 3 + 1$ implementation with the same modular-multiply extension, XRT improvements are smaller: 7,791,578 to 7,711,455 cycles at M1 (1.028%) and 11,049,301 to 10,522,959 at M8 (4.764%). These are KEM intervals, not the whole-launch values of Fig. 5.

Bank sizing is a separate implementation choice. CT pair butterflies do not need 32 independent products simultaneously, motivating a 16-multiplier implementation. A matched full-core post-route comparison reduces LUTs from 356,447 to 341,680, FFs from 272,430 to 270,865, and DSPs from 192 to 176; both close 250 MHz. The same application ELF changes complete-KEM cycles by at most 0.041% across SimX, processor RTL, and XRT at M1/M8 (Table II gives

TABLE II
NTT MAPPING AND BANK CONTROL, XRT KEM-INTERVAL CYCLES.
HISTORICAL MATCHED POINTER-ENABLED CORE; OTHER OPERATIONS
FIXED WITHIN THIS FAMILY.

NTT implementation	M1	M8
Shared transpose + NTTMUL	7,791,578	11,049,301
Pair + NTTMUL, 32 multipliers	7,711,455	10,522,959
Pair + NTTMUL, 16 multipliers	7,711,455	10,518,691

TABLE III
INDEPENDENT RV32IM POST-ROUTE IMPLEMENTATIONS, F/D DISABLED,
W8T32, FINAL HALF-BANK NTT; WNS IS IN NS AT 250 MHZ.

Core	LUT	FF	DSP	WNS
NTT only	220,950	131,798	112	+0.004
NTT + Stage	223,444	133,512	112	+0.004
NTT + Round	222,935	138,065	112	+0.007

the XRT points). This historical comparison includes floating-point hardware and Pointer; its absolute area is not combined with the integer-only results below. It supports the bank choice, not an additional application speedup claim.

E. Full-core area and integer-width sensitivity

Table III reports separate RV32IM implementations of the NTT-equipped core with each register Keccak extension enabled independently. All use Vivado 2025.1, the Alveo V80 part, OPT3, a 250 MHz constraint, and fresh full-core synthesis/place/route. All have zero routing errors, 133 BRAM tiles, and 112 DSPs. They are routed core implementations, not complete board shells or on-board measurements.

Stage adds 1.129% LUTs and 1.300% FFs over NTT only; Round adds 0.898% and 4.755%, respectively. Round’s deeper pipeline increases storage, but its integrated LUT count is slightly lower than Stage’s. Whole-core deltas include integration and placement effects; the two metrics do not select a universal area winner. The common runtime performance experiment enables all backends, whereas this table measures independent area configurations. We therefore do not combine them into a claimed measured throughput-per-area ranking. A matched integer-only Pointer implementation is not yet available for this table.

RV64 halves Keccak collective issues but widens more than the primitive. With F/D disabled on both cores, whole-launch retired instruction counts for the Stage/Round backends decline only 7.002%/5.730%. M8 XRT cycles change by $-0.403\%/+0.180\%$. The corresponding RV64 cores use 432,843/431,004 LUTs, about $1.94\times$ the RV32 cores; neither closes the 250 MHz target (WNS $-0.148/-0.013$ ns). These single-run implementation results favor keeping narrow NTT arithmetic independent of XLEN and evaluating width at complete core scope. They do not establish a general disadvantage of 64-bit GPUs.

V. RELATED WORK AND SCOPE

NTT-PIM maps operations according to the location of butterfly partners in memory buffers [10], and BP-NTT co-designs SRAM layout and modular multiplication [11]. Our storage and execution domain is the SIMT GPR bank, with explicit pair participation and normal per-lane writeback. RISQ-V already demonstrates CPU register-coupled NTT/Keccak acceleration [4]; Nannipieri et al. integrate scalar NTT arithmetic extensions into a RISC-V pipeline [5]. Register residence, modular multiplication, and butterfly fusion individually are prior ideas.

ML-Cube’s software Keccak layout and the NVIDIA collective-round patent are particularly close to our 25-lane execution boundary [6], [7]. Our contribution is its concrete RV32/RV64 operand and single-vector writeback realization together with communication-specific NTT and a controlled implementation study, not the first such collective concept. PacQ and SynGPU illustrate related GPU dataflow/hardware co-design for other workloads [12], [13]. CHAM and ZK-Flex motivate evaluating cryptographic acceleration beyond an individual transform [14], [15]. Their different algorithms and platforms preclude direct speedup comparisons.

The evidence here covers ML-KEM-768 at fixed known-answer inputs and two request counts. It does not establish optimal occupancy, input-distribution statistics, ML-DSA end-to-end acceleration, activity-based energy, or side-channel resistance. Constant arithmetic structure and KAT success are not substitutes for a security evaluation. Additional concurrency points, input diversity, and board execution would broaden the design conclusions.

VI. CONCLUSION

Ordinary warp registers can support both pairwise NTT butterflies and 25-lane Keccak collectives without introducing multi-destination writeback or persistent hidden accelerator state. Matching the operation to the communication structure enables a compact NTT bank and practical stage/round Keccak pipelines. Complete ML-KEM, however, bounds the useful fusion depth: large primitive gains shrink at application scope, and a pointer engine can lead at higher concurrency. State placement, instruction granularity, and request concurrency must therefore be evaluated together.

CONTENT GENERATED BY AI

OpenAI Codex assisted with manuscript drafting, literature organization, and figure-generation code. The draft uses archived experimental results; author review of the text, citations, and interpretation remains required.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [3] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [4] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [5] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [6] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [7] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [8] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [9] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>
- [10] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [11] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [12] R. Yin, Y. Li, and P. Panda, “PacQ: A SIMT microarchitecture for efficient dataflow in hyper-asymmetric GEMMs,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [13] Y. Yao, C. Zhang, C. Qi, R. Chen, J. Wang, Z. Fu, N. Jing, X. Liang, and Z. Song, “SynGPU: Synergizing CUDA and bit-serial tensor cores for vision transformer acceleration on GPU,” in *Proc. Design Automation Conf. (DAC)*, 2025.
- [14] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [15] A. Putra, C. M. Duong, A. Q. Pham, and J.-Y. Kim, “ZK-Flex: A flexible and scalable framework for accelerating zero-knowledge proofs,” in *Proc. Design Automation Conf. (DAC)*, 2026.